

SYSTEM DESIGN

Chapter 53: Design a Chat System

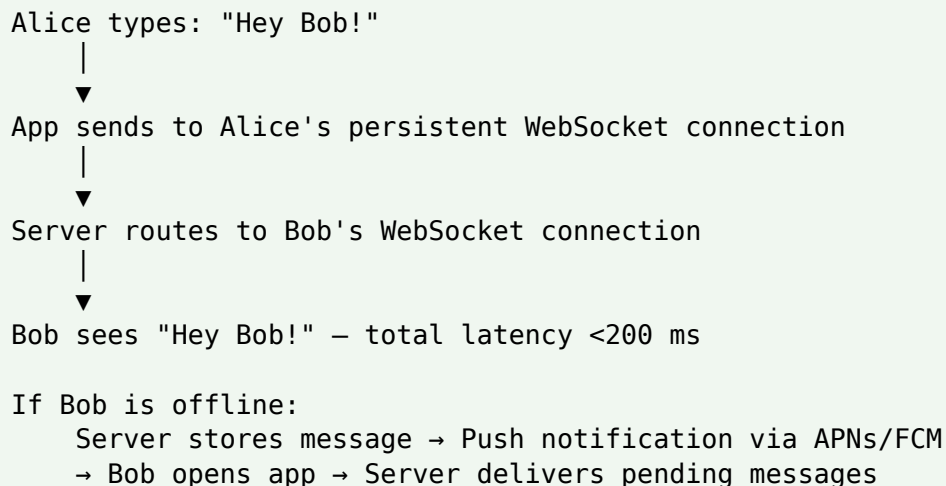
Personal Notes

URL shortener taught you the request-response model. News Feed added fan-out patterns and real-time push. Chat systems (WhatsApp, Slack, Discord, Messenger) go further: they're **PERSISTENTLY CONNECTED** — every user maintains a live socket to the server, waiting to receive messages the instant they arrive. This changes the architecture completely.

This chapter designs a WhatsApp/Slack-like chat system end-to-end. The core challenge isn't storing messages (that's easy) — it's **WEBSOCKET** at scale (100M concurrent connections), **DELIVERY GUARANTEES** (never lose a message, don't duplicate), and **GROUP CHAT** fan-out (1 message to 500 group members). Along the way we'll design the message-flow, tackle offline users with push notifications, and handle failure modes that just don't exist in stateless systems.

1. The Problem

Design a chat system that supports 1:1 messaging, group chats, real-time delivery, and offline notification. Users see messages the moment they arrive (subject to network latency). Messages are persisted so users can catch up after being offline, on a new device, or in message history.



2. Step 1 — Requirements Gathering

2.1 Functional Requirements

- Send/receive 1:1 messages in real-time
- Group chats (multiple recipients, up to N members)
- Show online/offline presence (last seen)

- Delivery + read receipts (sent, delivered, read)
- Push notifications when app is closed
- Message history across devices
- (Optional) Media (images, video, files), typing indicators, E2E encryption

2.2 Non-Functional Requirements

- Scale — assume 1B total users, 500M DAU, 60M concurrent connections at peak
- Latency — end-to-end message delivery p99 < 500 ms
- Availability — 99.99% uptime (people depend on chat)
- Durability — messages MUST NOT be lost, ever
- Consistency — messages must arrive in ORDER within a chat

2.3 Constraints to Confirm

- Max group size — 200? 5000? Determines fan-out cost
- Multi-device — can a user be logged in on 3 devices simultaneously?
- Message retention — forever, or expire after N days?
- E2E encryption — required? Restricts server-side features like search

The critical clarification: "Are messages ordered within a chat?" Almost always YES for chat. This drives the message storage design (sequence numbers per chat, monotonic increments). Confirming ordering upfront prevents rebuilding later when the interviewer asks about causal consistency.

3. Step 2 — Capacity Estimation

Messages

```
500M DAU × 40 messages/day = 20B messages/day
÷ 86,400 seconds/day
≈ 230K messages/sec average
× 3 peak
≈ 700K messages/sec peak (write QPS)
```

Concurrent Connections

500M DAU, ~12% online at once (peak) = 60M concurrent WebSockets

Per node: ~50-100K sockets is a well-tuned limit

Need: 60M / 75K = ~800 WebSocket gateway nodes at peak

This dwarfs everything else in the system — connection state is the biggest scaling challenge in chat.

Storage

20B messages/day × 500 bytes/msg (text + metadata)
= 10 TB/day of raw message data
× 5 years retention
= ~18 PB total

Compressed: ~30% smaller. Media stored separately in S3+CDN.

Push Notifications

Roughly 50-70% of messages go to offline recipients
→ ~150K push notifications/sec peak

Delivered via APNs (Apple), FCM (Google), WNS (Microsoft).
These are OUR problem to send, but their problem to deliver.

4. Step 3 — API Design

Chat APIs split into two paths — HTTP (persistent state) and WebSocket (real-time messages).

HTTP APIs

POST /api/chats	Create a chat (group or 1:1)
GET /api/chats	List user's chats
GET /api/chats/:chat_id/messages	Fetch message history (paginated)
POST /api/chats/:chat_id/members	Add user to group
POST /api/messages/:msg_id/read	Mark as read
POST /api/media/upload	Upload image/file → returns URL

WebSocket Events (client ↔ server)

```
{
  "type": "send_message",
  "chat_id": "abc123",
  "content": "Hello!",
  "client_msg_id": "uuid-1234"           // client-generated, for idempotency
}

// Server → client:
{
  "type": "message",                    // new incoming message
  "chat_id": "abc123",
  "sender_id": "alice",
  "content": "Hello!",
  "server_msg_id": 987654,              // monotonic per chat
  "timestamp": "..."
}

{
```

```

    "type": "delivery_ack",          // your message reached recipient
    "server_msg_id": 987654
  }

  {
    "type": "read_receipt",          // recipient read your message
    "server_msg_id": 987654,
    "read_by": "bob"
  }

  {
    "type": "typing",                // recipient is typing
    "chat_id": "abc123",
    "user_id": "bob"
  }

```

The client_msg_id is critical: It lets you achieve at-most-once semantics on writes. If a client sends a message, doesn't receive an ack (network flap), and retries — the server sees the same client_msg_id and returns the existing server_msg_id without creating a duplicate. Without this, dropped acks silently cause duplicate messages.

5. Step 4 — Data Model

```

Table: users
  user_id (PK) | username | last_seen | status

Table: chats
  chat_id (PK) | type ("1:1" | "group") | name | created_at

Table: chat_members
  chat_id | user_id | joined_at | role
PRIMARY KEY (chat_id, user_id)
INDEX on (user_id) — for "my chats" query

Table: messages
  chat_id | msg_seq (per-chat sequence) | sender_id | content
  | media_urls | server_ts | client_msg_id (unique per sender)
PRIMARY KEY (chat_id, msg_seq)
— msg_seq is monotonically increasing per chat: guarantees order

Table: delivery_status
  chat_id | msg_seq | user_id | status | status_at
status ∈ {"sent", "delivered", "read"}

```

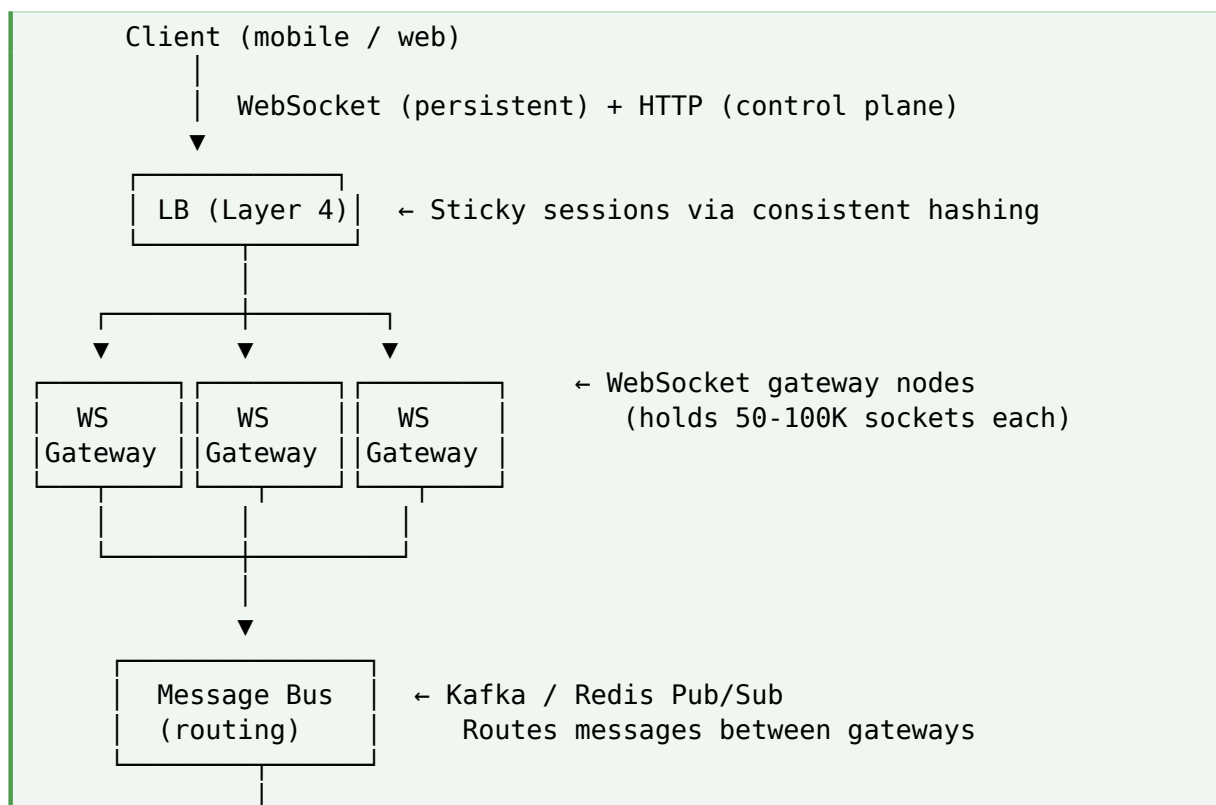
Per-chat sequence, NOT global: msg_seq is monotonic PER CHAT — chat A might be at seq 500, chat B at seq 12000. This makes ordering trivial (just sort by seq) and lets each chat's messages be co-located on one shard. A global counter would be a single point of contention at 700K writes/sec.

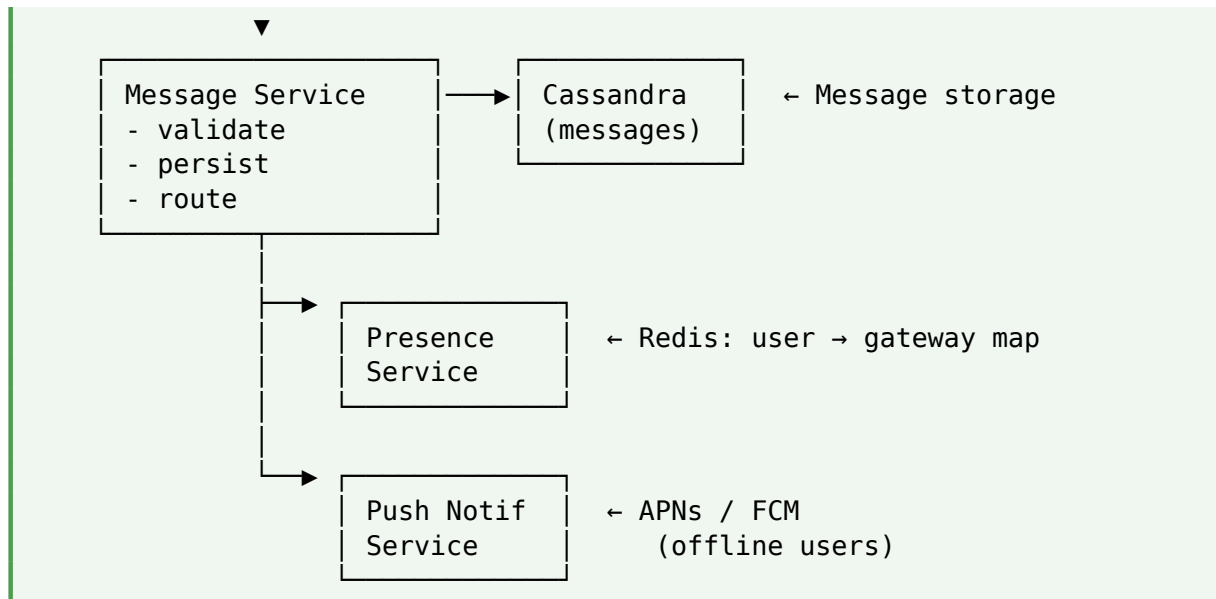
6. WebSocket vs HTTP — Why It Matters

Aspect	HTTP + Poll	WebSocket
Connection	Short-lived (per request)	Persistent (open for hours/days)
Direction	Client → server (request-response)	Bidirectional (server can push)
State	Stateless — LB routes any request	Stateful — user tied to specific server
Latency (message)	~200 ms round-trip + poll delay	~50 ms (server pushes immediately)
Overhead	New TCP + TLS every request	One handshake, then binary frames
Scale limit	Effectively unlimited (stateless)	Bound by concurrent-connection count per node

WebSocket is not the only option: Server-Sent Events (SSE) is a simpler alternative for server → client only. HTTP long polling still works if you can't use WebSocket (some corporate proxies block WS). For a modern chat app, WebSocket is the default; mention alternatives to show you know the trade-offs.

7. Step 5 — High-Level Architecture





Component Roles

- WebSocket Gateway — terminates client sockets, forwards events to backend
- Message Bus (Kafka/Redis) — routes messages between gateway nodes
- Message Service — validates, persists, orchestrates delivery
- Presence Service — tracks who's online + which gateway holds their socket
- Push Notification Service — sends APNs/FCM messages to offline users

8. Deep Dive: 1:1 Message Flow

The complete message journey from Alice to Bob:

1. Alice sends message via her WebSocket to Gateway-A
2. Gateway-A → Message Service:
 - Validate (spam check, auth)
 - Look up chat's shard, get next msg_seq
 - Persist to Cassandra
 - Return server_msg_id to Alice as ACK
3. Message Service → Presence Service:
 - "Where is Bob?" → returns Gateway-B (if online)
 - returns "OFFLINE" (if not)
- 4a. If Bob is ONLINE:
 - Message Service → Kafka topic for Gateway-B
 - Gateway-B receives event → pushes to Bob's WebSocket
 - Bob's client → sends delivery_ack back → propagated to Alice
- 4b. If Bob is OFFLINE:

Message Service → Push Notification Service
Push Service → APNs/FCM with notification payload
Bob's phone shows notification
When Bob opens app → GET /messages?chat_id=... → catches up

Why route via message bus (Kafka) instead of gateway-to-gateway direct calls: Gateways come and go (scaling, deploys). Direct connections between them would require complex service discovery + retry logic. A message bus decouples them — any gateway publishes to a topic, only the gateway with the recipient consumes. Loss-tolerant too: retries handled by Kafka's replay.

9. Deep Dive: The Presence Service

For every online user, we need to know WHICH gateway node holds their WebSocket. This lookup happens on EVERY message send — it has to be fast.

```
// Redis structure – user online → gateway mapping
Key: "presence:{user_id}"
Value: {
  "gateway_id": "gw-042",
  "connected_at": 1699500000,
  "device_ids": ["mobile-abc", "web-xyz"]    // multi-device
}
TTL: 60 seconds (refreshed by gateway heartbeat)

// On WebSocket connect:
gateway.on("connect", user => {
  redis.setex(`presence:${user}`, 60, gatewayInfo);
  redis.publish("user_online", user);
});

// On disconnect / TTL expiry:
gateway.on("disconnect", user => {
  redis.del(`presence:${user}`);
  redis.publish("user_offline", user);
});
```

Heartbeat Design

Client sends a heartbeat (ping) every ~20 seconds. Gateway refreshes the Redis TTL. If a client disconnects unexpectedly (phone dies, plane mode), the entry expires within 60 seconds — no explicit disconnect event needed.

Presence is expensive at scale: 1B users × 500M DAU × 12% concurrent = 60M presence entries. Redis handles it — but every message send does a Redis GET. At 700K messages/sec, that's 700K Redis reads/sec. Shard Redis by user_id hash across multiple instances. Also cache locally within Message Service (a hot user's location doesn't change every second).

10. Deep Dive: Delivery Guarantees

The Three Guarantees

Guarantee	Semantics	Trade-off
At-most-once	0 or 1 delivery	Fire-and-forget. Simple. Messages CAN be lost.
At-least-once	1 or more deliveries	Retry until ack'd. Duplicates possible; use idempotency to dedup.
Exactly-once	Exactly 1 delivery	Combines at-least-once + dedup on receiver. What chat needs.

How Chat Achieves Exactly-Once

1. Client generates a `client_msg_id` (UUID) before sending
2. Server dedupes on (`sender_id`, `client_msg_id`) — if already stored, return existing `server_msg_id`
3. Client keeps message in an outbox until `server_msg_id` ack received
4. Recipient dedupes on `server_msg_id` — never display same `server_msg_id` twice

Idempotency is the trick: "Exactly-once" networking is theoretically impossible (Two Generals Problem). What we actually get is at-least-once DELIVERY + idempotent SEMANTICS — retries are safe because duplicate operations don't cause duplicate effects. This is the pattern for anywhere you need reliability in distributed systems.

11. Deep Dive: Group Chat

1:1 chat is one recipient per message. Group chat means a single incoming message must be delivered to N recipients — anywhere from 3 to 5000+ members. This is where the design gets interesting.

Approach — Fan-Out at the Message Service

Alice sends message to group with 500 members



Message Service:

1. Persist ONCE to messages table
2. Query `chat_members` for group's members (~500)
3. For each member:
 - If online → publish to their gateway's Kafka topic
 - If offline → queue push notification
4. Return single ack to Alice

500 members → 500 delivery attempts, but ONE stored copy.

Latency: still <500ms since fan-out is parallelizable.

Scaling Concerns

- Small groups (< 200 members) — trivial. Handle fan-out synchronously.
- Medium groups (200-5000) — fan out via Kafka asynchronously. Message Service returns fast; delivery happens in parallel.
- Massive groups (10K+, like Slack Enterprise channels) — treat like feeds: users pull messages when active, don't push to everyone.

The Slack channel scaling problem: A Slack channel with 50,000 members receiving a message would require 50K parallel pushes and 50K DB writes for delivery status. Slack's approach: don't track per-user delivery status for massive channels; users fetch unread state on channel open. Trade a per-user "unread" state for scalability.

12. Deep Dive: Message Storage & Sharding

Sharding Strategy

Shard the messages table by chat_id (not by user_id). This co-locates every message in a chat on one shard, so fetching history for a chat is a single-shard query.

Shard key: $\text{hash}(\text{chat_id}) \% N$

Chat "abc123" always goes to shard 5.
ALL messages for that chat live on shard 5.
Reads (message history) hit only shard 5.
Writes are load-balanced across shards evenly.

Alternative: shard by user_id — WRONG.
A single message is between users on different shards.
Cross-shard writes on every message. Bad.

Why Cassandra

- Write-heavy (700K/sec) — Cassandra excels here
- Access pattern is (chat_id, msg_seq range) — perfect for wide-column with clustering
- Horizontal scale-out — add nodes as data grows
- Time-series-like data — Cassandra was literally designed for this

Message Retention

Not all messages live forever. Use TTL on the messages table (Cassandra native) — say 5 years. For "disappearing messages" (Signal, WhatsApp) use much shorter TTLs. Media in S3 has its own retention policy.

13. Deep Dive: Multi-Device Sync

Alice is logged in on phone AND laptop. She sends a message from her phone. Her laptop should show it too. And when Bob replies, BOTH devices should receive it.

Approach

5. Each device gets its own WebSocket connection
6. Presence Service maps (user_id → list of gateways/devices), not just one
7. Message Service fans out to EVERY device the user has online
8. Each device tracks its own "last read msg_seq" per chat

```
// Presence entry – multiple devices
{
  "gateway_id_by_device": {
    "phone-abc": "gw-042",
    "laptop-xyz": "gw-108"
  }
}

// On new message, Message Service fans out to both gateways.
// Each device processes and displays independently.
```

Sync gap on reconnect: Device connects → server compares its last_seen_msg_seq to current server state → server sends missed messages. This is why msg_seq is monotonic per chat — clients can request "everything after seq X" in one shot.

14. Deep Dive: Push Notifications

When a user is OFFLINE (app closed, phone locked), they can't receive messages via WebSocket. Push notifications via APNs (iOS) or FCM (Android) wake them up.

Message sent to Bob, but Bob is offline.

1. Message Service publishes to a "push" Kafka topic
2. Push Service consumes the topic
3. Looks up Bob's device tokens (registered on install)
4. Constructs platform-specific payload
5. Sends to APNs (iOS) or FCM (Android)
6. Cloud service delivers to Bob's device → shows notification

Bob taps notification → app opens → WebSocket reconnects
→ syncs missed messages via GET /messages

Push notifications are best-effort: APNs/FCM don't guarantee delivery. Devices may be in airplane mode, powered off, out of coverage. Never rely on push for critical delivery — always deliver via WebSocket on next connect too. Push is UX ("you have a new message"), not the transport layer.

15. Additional Features

15.1 Typing Indicators

Send "typing" WebSocket events without persisting to DB. Auto-expire after ~5 seconds. High volume + low value → keep the volume off your storage.

15.2 Read Receipts

Client sends read event when message is displayed → server records in `delivery_status` → notifies sender. For group chats: aggregate reads ("read by 3 of 5") to avoid N^2 receipt traffic.

15.3 Media Handling

Same as feeds: client uploads to S3 via presigned URL, message contains only the URL. NEVER put media bytes in the message body. Thumbnails generated async. Video transcoded to multiple resolutions. CDN serves globally.

15.4 End-to-End Encryption

Signal Protocol (used by Signal, WhatsApp, iMessage). Keys negotiated pairwise (or per group session). Server sees only encrypted blobs — no ability to read content or search server-side. Trade-off: server-side search, cloud backup, message recovery all become much harder.

15.5 Message Search

Without E2E: index messages in Elasticsearch as a background job. With E2E: search must happen ON DEVICE — client downloads full history and searches locally. This is why Signal search is slower than iMessage search.

16. Failure Modes

Failure	Mitigation
Gateway node crashes	Clients auto-reconnect via LB to different gateway. Presence updates. Some sockets briefly dropped.
Message Service down	Writes fail. Client retries with <code>client_msg_id</code> — safe. UX: "sending..." indicator.
Kafka down	Messages queue up on Message Service. Deliver from disk backlog when Kafka recovers.
Push notification fails	Best-effort — user sees message on next foreground. No retries needed.
Presence data stale	Message routed to wrong gateway → Kafka fan-out failure detected →

Failure	Mitigation
	fallback to push notification.
Cassandra shard down	That chat's history unavailable temporarily. Reads from replica. Writes retry.
Client offline for weeks	Bounded catch-up (last 30 days). Older messages fetched paginated via API.

17. Trade-offs Summary

Decision	Trade-off
WebSocket vs long polling	WebSocket = lower latency; polling = simpler, works through more networks
Global vs per-chat msg_seq	Per-chat = trivial to scale; global = simpler but single point of contention
Sync group fan-out vs async	Sync = simpler but caps group size; async = scales but complex retry
Delivery status per-user or not	Per-user = rich UX; skip for massive channels to save writes
Push notif for every msg or bundled	Every = real-time; bundled = friendlier, less battery drain
Server-side vs on-device search	Server = fast; on-device required if E2E encrypted
Retain messages forever or TTL	Forever = simple UX; TTL = compliance-friendly, saves storage
E2E encryption yes/no	Security vs features (search, backup, cloud sync all harder)

18. Common Mistakes

Mistake 1: Ignoring the WebSocket state challenge

Saying "we'll use WebSockets" without addressing the 60M concurrent connections problem. Discuss connection distribution, gateway sizing, sticky routing, heartbeats — this is HALF the design.

Mistake 2: Using global msg_seq

A global monotonic counter for message IDs becomes a single point of contention at 700K writes/sec. Use PER-CHAT sequences instead — same ordering guarantee, trivially distributable.

Mistake 3: Skipping idempotency

Networks flap. Clients retry. If your server treats every send as a new message, one retry = duplicate. Always use `client_msg_id` + server dedup on `(sender_id, client_msg_id)`.

Mistake 4: Direct gateway-to-gateway routing

If Gateway-A holds Alice and Gateway-B holds Bob, it's tempting to have A call B directly. But now every gateway must know every other gateway's location. A message bus (Kafka) decouples them — publish to a topic, whichever gateway has the recipient consumes it.

Mistake 5: No plan for offline users

Bob is offline → server must persist message + send push notification. Missing this leaves offline users with silent, missed messages. Push is best-effort — always combined with WebSocket sync on reconnect.

Mistake 6: Treating groups like $N \times 1:1$ messages

Storing the same message once per group member is $500\times$ the storage for a 500-member group. Store ONCE, fan out delivery. Only DELIVERY STATUS is per-user.

Mistake 7: Not addressing multi-device

Users have phone + laptop + desktop. All should receive messages. Presence Service must track (user → list of connections), not just one connection per user. Fan-out to all their devices.

19. Best Practices

- Use WebSocket for real-time delivery — accept the stateful complexity
- Use `client_msg_id` for at-most-once semantics on writes
- Shard messages by `chat_id` — co-locates history on one shard
- Use per-chat `msg_seq` (not global) for ordering
- Route between gateways via message bus (Kafka/Redis Pub-Sub), never direct RPC
- Track presence in Redis with TTL + heartbeat — self-healing
- Combine WebSocket delivery + push notifications for offline users
- Fan out group messages async via Kafka — one store, N deliveries
- Support multi-device by mapping user → list of gateways
- For massive channels (10K+ members), skip per-user delivery status

20. Quick Reference

Concept	Meaning
WebSocket	Persistent bidirectional connection over TCP

Concept	Meaning
Gateway	WS server terminating client sockets, forwarding events
Presence Service	Redis lookup of user → gateway/device mapping
Message Bus	Kafka/Redis Pub-Sub routing messages between gateways
client_msg_id	Client-generated UUID for idempotent writes
server_msg_id (msg_seq)	Monotonic per-chat sequence — guarantees ordering
Delivery guarantees	At-most-once, at-least-once, exactly-once (via dedup)
Fan-out	Group chat: 1 store, N delivery attempts
APNs / FCM	Push notification services for iOS/Android
Heartbeat	Client ping to refresh presence TTL
Sticky routing	LB routes to specific gateway via consistent hashing
Cassandra clustering	Rows partitioned by chat_id, clustered by msg_seq
Multi-device sync	User → list of gateways; fan out to each device

21. Related Interview Problems

The chat system pattern powers many real-time products. Once you've mastered this:

- Design Slack — same core + workspaces, channels, threads, integrations
- Design Discord — same core + voice channels (WebRTC), server tiers
- Design a Live Comments overlay — one-to-many chat during video streams
- Design a Collaborative Editing tool (Google Docs) — WebSocket + OT/CRDT algorithms
- Design a Multiplayer Game Server — WebSocket + game state sync, tick-based updates
- Design a Real-Time Auction / Trading — WebSocket + priority queues, order matching
- Design a Video Conferencing System (Zoom) — WebRTC + signaling servers
- Design a Notification System — cross-channel delivery, priority, batching

22. Final Takeaway

Chat systems are the first design that requires STATEFUL infrastructure — the WebSocket pool is a first-class citizen of the architecture. Most first-time candidates focus on the message storage (which is actually the easy part) and skip the harder problems: distributing 60M persistent connections, routing messages between gateways, achieving exactly-once semantics, and handling offline users. Nail those four and you've cracked the design.

For interviews, memorize the flow: WebSocket + gateway architecture → presence service → message bus (Kafka) for routing → per-chat msg_seq → client_msg_id for idempotency → group fan-out → push notifications for offline. If you can walk this in 35 minutes, you're ready for any chat-adjacent design (Slack, Discord, collaborative editing, live comments).

Key Takeaway: Chat = STATEFUL system. WEBSOCKET gateways hold persistent connections (50-100K per node). PRESENCE SERVICE maps user → gateway. Messages route between gateways via MESSAGE BUS (Kafka/Redis Pub-Sub). Achieve EXACTLY-ONCE via client_msg_id + server dedup. Store messages once, fan out delivery to N recipients. Per-chat MSG_SEQ (not global) guarantees ordering. Combine WebSocket + PUSH NOTIFICATIONS for offline users. Shard by chat_id in Cassandra. Multi-device via user → list of gateways. E2E encryption trades server features for privacy.

23. What's Next?

With chat covered, the third canonical system design is done. Next up:

- Design a Rate Limiter — token bucket at distributed scale, coordination challenges
- Design a Distributed Cache — Redis internals, consistent hashing, replication
- Design Uber / Lyft — geohashing, driver-rider matching, real-time location
- Design YouTube — video encoding pipeline, adaptive bitrate, CDN edge
- Design a Notification System — cross-channel delivery, priorities, templates
- Design a Search Autocomplete — trie-based, prefix caching, popularity scoring
- Deep dive: WebRTC — how voice/video actually works (peer-to-peer, TURN servers)
- Deep dive: Kafka internals — partitions, consumer groups, exactly-once semantics